Lab-1 : SQL Commands

Login and Initialization

1. Connect to MySQL/MariaDB

mysql -u root -p

Explanation:

- `mysql`: MySQL/MariaDB command-line client
- `-u root`: Specifies the username (root in this case)
- `-p`: Prompts for password entry
- After entering the correct password, you'll enter the MariaDB monitor

Output: You'll see the welcome message showing:

- Server version (e.g., 10.4.32-MariaDB)
- Connection ID
- Commands available (`help;` or `\h`)

2. Exit MySQL/MariaDB

Bye
-- or
exit
-- or
quit
-- or
\q

Explanation: Any of these commands will close the MySQL/MariaDB session and return you to the command prompt.

Important Note

SQL commands must be executed inside the MySQL monitor (after login).

- Commands like `CREATE DATABASE`, `USE`, `SELECT` only work inside the MySQL monitor

- If you try to run them directly in the command prompt (without logging in first), you'll get the error: `'CREATE' is not recognized as an internal or external command`

Database Operations

1. Show All Databases

show databases;

Explanation: Displays a list of all databases available on the MySQL/MariaDB server.

2. Create Database

create database CSE370Lab01;

Explanation: Creates a new database named "CSE370Lab01". Database names are case-sensitive on some systems.

3. Use Database

use CSE370Lab01;

Explanation: Selects the database to work with. All subsequent operations will be performed on this database.

Table Operations

4. Create Table

```
create table Lab_Grades (
    std_id char(4),
    name varchar(30),
    major char(3),
    section char(1),
    days_present int,
    project_marks double,
    cgpa decimal(3,2),
    submission_date date
```

);

Explanation: Creates a new table with specified columns and data types:

- `char(n)`: Fixed-length string
- `varchar(n)`: Variable-length string
- `int`: Integer number
- `double`: Decimal number with floating point
- `decimal(3,2)`: Decimal with 3 total digits, 2 after decimal point
- `date`: Date value (YYYY-MM-DD format)

5. Show Tables

show tables;

Explanation: Lists all tables in the currently selected database.

6. Describe Table Structure

describe Lab_Grades;

Explanation: Shows the structure of a table including column names, data types, and constraints.

Altering Table Structure

7. Add Column

alter table Lab_Grades add project_title char(10);

Explanation: Adds a new column named "project_title" to the existing table.

8. Modify Column

alter table Lab_Grades modify column project_title varchar(50);

Explanation: Changes the data type of an existing column from `char(10)` to `varchar(50)`.

9. Drop Column

```
alter table Lab_Grades drop column project_title;
```

Explanation: Removes a column from the table permanently.

10. Rename Column

```
alter table Lab_Grades change column submission_date sub_date date;
```

Explanation: Renames a column from "submission_date" to "sub_date". You must specify the data type even when just renaming.

Data Manipulation

11. Insert Data

```
insert into Lab_Grades values
('s001', 'Abir', 'CS', '1', 10, 18.5, 3.91, '2018-09-15'),
('s002', 'Nafis', 'CSE', '1', 12, 20, 3.86, '2018-08-15');
```

Explanation: Inserts multiple rows of data into the table. Values must match the column order and data types.

12. Update Data

```
update lab_grades set major = 'CSE' where name = 'Arafat';
```

Explanation: Updates existing records. Changes the "major" value to 'CSE' for the student named 'Arafat'. Always use WHERE clause to avoid updating all rows.

```
update lab_grades set name = 'Naheed' where std_id = 's004';
```

Explanation: Updates the name based on student ID condition.

13. Delete Data

```
delete from Lab_Grades where name = 'Naima';
```

Explanation: Deletes specific rows matching the condition. Without WHERE, it would delete all rows!

delete from Lab_Grades where days_present < 8;

Explanation: Deletes all records where days_present is less than 8.

Query Operations (SELECT)

14. Select All Data

select * from Lab_Grades;

Explanation: Retrieves all columns and all rows from the table. The asterisk (*) means "all columns".

15. Select Specific Columns

select std_id, name, project_marks from lab_grades;

Explanation: Retrieves only the specified columns for all rows.

16. Select with Condition

select std_id, name, project_marks from lab_grades where project_marks > 15;

Explanation: Retrieves data only for rows where project_marks is greater than 15.

Calculations and Functions

17. Arithmetic Operations

select std_id, name, project_marks + days_present/12*5 from lab_grades;

Explanation: Performs calculation using the formula: `project_marks + (days_present/12)*5`. Follows standard mathematical operator precedence.

18. Round Function with Alias

select std_id, name, round(project_marks + days_present/12*5, 2) as lab_marks

from lab_grades;

Explanation:

- `round(value, 2)`: Rounds the result to 2 decimal places
- `as lab_marks`: Creates an alias (column name) for the calculated result

19. String Functions

select upper(name), lower(name) from lab_grades;

Explanation:

- `upper()`: Converts string to uppercase
- `lower()`: Converts string to lowercase

Sorting and Filtering

20. Distinct Values

select distinct major from lab_grades;

Explanation: Returns only unique values, removing duplicates from the result.

21. Order By (Ascending)

select * from Lab_Grades order by cgpa;

Explanation: Sorts results in ascending order by CGPA (lowest to highest). ASC is the default.

22. Order By (Descending)

select * from Lab_Grades order by cgpa desc;

Explanation: Sorts results in descending order (highest to lowest).

select * from Lab_Grades order by project_marks asc;
select * from Lab_Grades order by sub_date asc;

Explanation: Multiple examples sorting by different columns.

Date Functions

23. Extract Date Parts

select std_id, name, month(sub_date), year(sub_date)
from Lab_Grades order by sub_date asc;

Explanation:

- `month()`: Extracts the month number (1-12) from a date
- `year()`: Extracts the year from a date

select std_id, name, date(sub_date), month(sub_date), year(sub_date)
from Lab_Grades order by sub_date asc;

Explanation: `date()` returns the date part from a datetime value.

Range and Set Operations

24. Between Operator

select * from Lab_Grades where project_marks between 16 and 18;

Explanation: Selects rows where project_marks is between 16 and 18 (inclusive). Equivalent to `>= 16 AND <= 18`.

select * from Lab_Grades where sub_date between '2018-09-01' and '2018-09-30';

Explanation: BETWEEN also works with dates to find records within a date range.

25. IN Operator

select * from Lab_Grades where major in('CS','CSE');

Explanation: Matches rows where major is either 'CS' or 'CSE'. More concise than using multiple OR conditions.

26. OR Operator

```
select * from Lab_Grades where major = 'CS' or major = 'CSE';
```

Explanation: Alternative to IN operator, achieves the same result using logical OR.

Date Filtering

27. Filter by Month and Year

```
select * from Lab_Grades
where month(sub_date) = 9 and year(sub_date) = 2018;
```

Explanation: Finds all records submitted in September 2018 by extracting and comparing month and year values.

Pattern Matching (LIKE)

28. Starts With Pattern

```
select * from Lab_Grades where name like 'A%';
```

Explanation:

- % is a wildcard matching any number of characters
- Finds names starting with 'A'

29. Ends With Pattern

```
select * from Lab_Grades where name like '%m';
```

Explanation: Finds names ending with 'm'.

```
select * from Lab_Grades where name like '%neem';
```

Explanation: Finds names ending with 'neem'.

30. Contains Pattern

select * from Lab_Grades where name like '%i%';

Explanation: Finds names containing the letter 'i' anywhere.

31. Position-Specific Pattern

select * from Lab_Grades where name like '_b%';

Explanation:

- _ (underscore) matches exactly one character
- Finds names where 'b' is the second character

select * from Lab_Grades where name like '%d_';

Explanation: Finds names where 'd' is the second-to-last character.

32. Multiple Occurrences

select * from Lab_Grades where name like '%a%a%';

Explanation: Finds names containing at least two 'a's.

select * from Lab_Grades where name like '%e%e%';

Explanation: Finds names containing at least two 'e's.

Summary of Wildcards

- % - Matches zero or more characters
- _ - Matches exactly one character

Summary of Operators

- = - Equal to

- `>` - Greater than
- `<` - Less than
- `>=` - Greater than or equal to
- `<=` - Less than or equal to
- `between ... and ...` - Within a range (inclusive)
- `in(...)` - Matches any value in a list
- `like` - Pattern matching with wildcards
- `or` - Logical OR
- `and` - Logical AND

Lab-2 : SQL Aggregate Functions & Subqueries Reference

Aggregate Functions

1. MAX - Find Maximum Value

select max(project_marks) from lab_grades;

Explanation: Returns the highest value in the `project_marks` column. In this case, it returns 20.

Output:

```
+--------------------+
| max(project_marks) |
+--------------------+
|                 20 |
+--------------------+
```

---

2. SUM - Calculate Total

select sum(project_marks) from lab_grades;

Explanation: Adds up all values in the `project_marks` column. Returns 129.5 as the total of all project marks.

Output:

```
+--------------------+
| sum(project_marks) |
+--------------------+
|              129.5 |
+--------------------+
```

---

3. COUNT - Count Rows

select count(*) from lab_grades;

Explanation: Counts the total number of rows in the table. The asterisk (*) means count all rows. Returns 7 rows.

Output:

```
+----------+
| count(*) |
+----------+
|        7 |
+----------+
```

---

4. MIN - Find Minimum Value

select min(cgpa) from lab_grades;

Explanation: Returns the lowest value in the `cgpa` column. In this case, it returns 3.25.

Output:

```
+-----------+
| min(cgpa) |
+-----------+
|      3.25 |
+-----------+
```

---

5. AVG - Calculate Average

```
select major, avg(cgpa) as avg_cgpa
from lab_grades
where days_present > 10
group by major
having avg(cgpa) > 3.5
order by avg_cgpa desc
limit 1;
```

Explanation:

- `avg(cgpa)`: Calculates the average CGPA
- `as avg_cgpa`: Renames the result column
- `where days_present > 10`: Filters students with more than 10 days present
- `group by major`: Groups results by major
- `having avg(cgpa) > 3.5`: Filters groups where average CGPA is greater than 3.5 (HAVING is used after GROUP BY)
- `order by avg_cgpa desc`: Sorts by average CGPA in descending order
- `limit 1`: Returns only the top result

Output:

```
+-------+----------+
| major | avg_cgpa |
+-------+----------+
| CS    | 4.000000 |
+-------+----------+
```

GROUP BY Clause

6. Group By with MIN

```
select min(cgpa) from lab_grades group by major;
```

Explanation: Groups the data by `major` and then finds the minimum CGPA for each group. Returns 3 results (one for each major: CS, CSE, ECE).

Output:

```
+-----------+
| min(cgpa) |
+-----------+
|      3.57 |  -- CS minimum
|      3.70 |  -- CSE minimum
|      3.25 |  -- ECE minimum
+-----------+
```

---

7. Group By with Single Column

select cgpa from lab_grades group by major;

Explanation: Groups by major and shows one CGPA value per group (typically the first one encountered). Not very useful without an aggregate function.

Output:

```
+------+
| cgpa |
+------+
| 3.91 |
| 3.86 |
| 3.25 |
+------+
```

---

8. Group By with All Columns

select * from lab_grades group by major;

Explanation: Groups by major and shows one complete row per major (the first row in each group). Returns 3 rows, one for each major.

Output:

```
+--------+-------+-------+---------+--------------+---------------+------+-----------------+
| std_id | name  | major | section | days_present | project_marks | cgpa | submission_date |
+--------+-------+-------+---------+--------------+---------------+------+-----------------+
| s001   | Abir  | CS    | 1       |           10 |          18.5 | 3.91 | 2018-09-15      |
| s002   | Nafis | CSE   | 1       |           12 |            20 | 3.86 | 2018-08-15      |
```

```
| s004   | Nahid | ECE   | 2       |           7 |         16.5 | 3.25 | 2018-08-20      |
+--------+-------+-------+---------+-------------+--------------+------+----------------+
```

---

9. Group By with COUNT

select major, count(*) from lab_grades group by major;

Explanation: Groups data by major and counts how many students are in each major. Shows both the major name and the count.

Output:

```
+-------+----------+
| major | count(*) |
+-------+----------+
| CS    |        3 |
| CSE   |        2 |
| ECE   |        2 |
+-------+----------+
```

Subqueries

10. Subquery with Comparison

select name from lab_grades
where cgpa = (select max(cgpa) from lab_grades);

Explanation:

- Inner query: `select max(cgpa) from lab_grades` finds the maximum CGPA (4.00)
- Outer query: Finds the name of the student(s) whose CGPA equals that maximum value
- This is a **scalar subquery** (returns a single value)

Output:

```
+--------+
| name   |
+--------+
| Arafat |
```

```
+--------+
```

---

11. Subquery with ANY

```
select * from lab_grades
where major = 'CSE'
and cgpa > any (select cgpa from lab_grades where major = 'CS');
```

Explanation:

- Inner query: Gets all CGPA values for CS major students (3.91, 3.57, 4.00)
- `> any`: Returns true if the value is greater than **at least one** value in the subquery result
- Finds CSE students whose CGPA is greater than at least one CS student's CGPA

Output:

```
+--------+---------+-------+---------+--------------+---------------+------+-----------------+
| std_id | name    | major | section | days_present | project_marks | cgpa | submission_date |
+--------+---------+-------+---------+--------------+---------------+------+-----------------+
| s002   | Nafis   | CSE   | 1       |           12 |            20 | 3.86 | 2018-08-15      |
| s006   | Tasneem | CSE   | 1       |           12 |          17.5 | 3.70 | 2018-08-15      |
+--------+---------+-------+---------+--------------+---------------+------+-----------------+
```

---

12. Subquery with ALL

```
select * from lab_grades
where major = 'CSE'
and cgpa > all (select cgpa from lab_grades where major = 'CS');
```

Explanation:

- Inner query: Gets all CGPA values for CS major students (3.91, 3.57, 4.00)
- `> all`: Returns true only if the value is greater than **every** value in the subquery result
- Finds CSE students whose CGPA is greater than all CS students' CGPA
- Returns empty because no CSE student has CGPA > 4.00 (the highest CS CGPA)

Output:

Empty set (0.001 sec)

---

13. Correlated Subquery with EXISTS

```
select name from lab_grades l1
where exists (
   select cgpa from lab_grades l2
   where l1.cgpa > l2.cgpa
   and l1.major = l2.major
   and l1.std_id != l2.std_id
);
```

Explanation:

- This is a **correlated subquery** - the inner query references the outer query's table (`l1`)
- `l1` and `l2` are **table aliases** for the same table
- `exists`: Returns true if the subquery returns at least one row
- Finds students who have a higher CGPA than at least one other student in the same major
- Logic: For each student (l1), check if there exists another student (l2) in the same major with a lower CGPA

Output:

```
+---------+
| name    |
+---------+
| Arafat  |  -- CS major with CGPA 4.00 (higher than Abir 3.91 and Tasneem 3.57)
| Abir    |  -- CS major with CGPA 3.91 (higher than Tasneem 3.57)
| Muhtadi |  -- ECE major with CGPA 3.67 (higher than Nahid 3.25)
| Nafis   |  -- CSE major with CGPA 3.86 (higher than Tasneem 3.70)
+---------+
```

Summary of Key Concepts

Aggregate Functions

- `max()` - Maximum value
- `min()` - Minimum value
- `sum()` - Total sum
- `avg()` - Average value

- `count()` - Count rows

GROUP BY

- Groups rows that have the same values
- Used with aggregate functions
- `having` clause filters groups (used after `group by`)
- `where` clause filters rows (used before `group by`)

Subqueries

- **Scalar subquery**: Returns single value (used with `=`, `>`, `<`, etc.)
- **ANY**: True if comparison is true for at least one value
- **ALL**: True if comparison is true for all values
- **EXISTS**: True if subquery returns at least one row
- **Correlated subquery**: Inner query references outer query

Query Execution Order

1. `from` - Specify table
2. `where` - Filter rows
3. `group by` - Group rows
4. `having` - Filter groups
5. `select` - Choose columns
6. `order by` - Sort results
7. `limit` - Limit number of results

Lab-3 : SQL Joins

1. INNER JOIN (Explicit)

Basic INNER JOIN

```
-- Returns only matching rows from both tables
select t1.column1, t2.column2
from table1 t1
inner join table2 t2 on t1.id = t2.id;
```

Example with Student and Hobbies

```
select t1.name, t2.hobby
from students t1
inner join hobbies t2 on t1.id = t2.id;
```

Multiple INNER JOINs

```
select t1.name, t2.hobby, t3.course, t4.grade
from students t1
inner join hobbies t2 on t1.id = t2.id
inner join courses t3 on t2.id = t3.student_id
inner join grades t4 on t3.id = t4.course_id;
```

INNER JOIN with WHERE

```
select t1.name, t2.hobby
from students t1
inner join hobbies t2 on t1.id = t2.id
where t1.age > 20;
```

INNER JOIN with Multiple Conditions

```
select t1.name, t2.hobby
from students t1
inner join hobbies t2 on t1.id = t2.id and t1.department = t2.department;
```

INNER JOIN with Aggregate Functions

```
select t1.department, count(t2.hobby) as hobby_count
from students t1
inner join hobbies t2 on t1.id = t2.id
group by t1.department;
```

---

2. LEFT JOIN (Explicit)

Basic LEFT JOIN

```
-- Returns all rows from left table, matching rows from right table
-- NULL for non-matching rows from right table
select t1.column1, t2.column2
```

```
from table1 t1
left join table2 t2 on t1.id = t2.id;
```

Example with Student and Hobbies

```
-- Shows all students, even those without hobbies
select t1.name, t2.hobby
from students t1
left join hobbies t2 on t1.id = t2.id;
```

LEFT JOIN (alternative syntax)

```
select t1.name, t2.hobby
from students t1
left outer join hobbies t2 on t1.id = t2.id;
-- LEFT JOIN and LEFT OUTER JOIN are the same
```

Multiple LEFT JOINs

```
select t1.name, t2.hobby, t3.course, t4.grade
from students t1
left join hobbies t2 on t1.id = t2.id
left join courses t3 on t2.id = t3.student_id
left join grades t4 on t3.id = t4.course_id;
```

LEFT JOIN with WHERE (Filter Left Table)

```
-- Filter BEFORE join
select t1.name, t2.hobby
from students t1
left join hobbies t2 on t1.id = t2.id
where t1.age > 20;
```

LEFT JOIN with WHERE (Find NULL)

```
-- Find students WITHOUT hobbies
select t1.name
from students t1
left join hobbies t2 on t1.id = t2.id
where t2.id is null;
```

LEFT JOIN with COALESCE

```
-- Replace NULL with default value
select t1.name, coalesce(t2.hobby, 'No Hobby') as hobby
from students t1
left join hobbies t2 on t1.id = t2.id;
```

---

3. RIGHT JOIN (Explicit)

Basic RIGHT JOIN

```
-- Returns all rows from right table, matching rows from left table
-- NULL for non-matching rows from left table
select t1.column1, t2.column2
from table1 t1
right join table2 t2 on t1.id = t2.id;
```

Example with Student and Hobbies

```
-- Shows all hobbies, even those without students
select t1.name, t2.hobby
from students t1
right join hobbies t2 on t1.id = t2.id;
```

RIGHT JOIN (alternative syntax)

```
select t1.name, t2.hobby
from students t1
right outer join hobbies t2 on t1.id = t2.id;
-- RIGHT JOIN and RIGHT OUTER JOIN are the same
```

Multiple RIGHT JOINs

```
select t1.name, t2.hobby, t3.course, t4.grade
from students t1
right join hobbies t2 on t1.id = t2.id
right join courses t3 on t2.id = t3.student_id
right join grades t4 on t3.id = t4.course_id;
```

RIGHT JOIN with WHERE

```
select t1.name, t2.hobby
from students t1
right join hobbies t2 on t1.id = t2.id
where t2.hobby like 'S%';
```

RIGHT JOIN - Find Orphans

```
-- Find hobbies WITHOUT students
select t2.hobby
from students t1
right join hobbies t2 on t1.id = t2.id
where t1.id is null;
```

---

4. FULL OUTER JOIN

FULL OUTER JOIN (MySQL Workaround)

```
-- MySQL doesn't support FULL OUTER JOIN directly
-- Workaround using UNION
select t1.name, t2.hobby
from students t1
left join hobbies t2 on t1.id = t2.id
union
select t1.name, t2.hobby
from students t1
right join hobbies t2 on t1.id = t2.id;
```

FULL OUTER JOIN (PostgreSQL/SQL Server)

```
-- Returns all rows from both tables
-- NULL where no match exists
select t1.name, t2.hobby
from students t1
full outer join hobbies t2 on t1.id = t2.id;
```

---

5. CROSS JOIN (Explicit)

Basic CROSS JOIN

```
-- Cartesian product - every row from table1 with every row from table2
select t1.name, t2.hobby
from students t1
cross join hobbies t2;
```

CROSS JOIN with WHERE

```
select t1.name, t2.hobby
from students t1
cross join hobbies t2
where t1.department = 'CS';
```

---

6. SELF JOIN

SELF JOIN (Join table to itself)

```
-- Find employees and their managers
select e1.name as employee, e2.name as manager
from employees e1
inner join employees e2 on e1.manager_id = e2.id;
```

SELF JOIN - Find Pairs

```
-- Find students in the same department
select t1.name as student1, t2.name as student2, t1.department
from students t1
inner join students t2 on t1.department = t2.department
where t1.id < t2.id;  -- Avoid duplicates
```

SELF JOIN with LEFT JOIN

```
-- Include employees without managers
select e1.name as employee, e2.name as manager
from employees e1
left join employees e2 on e1.manager_id = e2.id;
```

---

7. IMPLICIT JOIN (Old Style - Using WHERE)

Basic Implicit INNER JOIN

```
-- Two tables
select t1.name, t2.hobby
from students t1, hobbies t2
where t1.id = t2.id;
```

Implicit JOIN - Multiple Tables

```
-- Three tables
select t1.name, t2.hobby, t3.course
from students t1, hobbies t2, courses t3
where t1.id = t2.id
and t2.id = t3.student_id;
```

Implicit JOIN - Four Tables

```
-- Four tables
select t1.name, t2.hobby, t3.course, t4.grade
from students t1, hobbies t2, courses t3, grades t4
where t1.id = t2.id
and t2.id = t3.student_id
and t3.id = t4.course_id;
```

Implicit JOIN with Additional WHERE Conditions

```
select t1.name, t2.hobby
from students t1, hobbies t2
where t1.id = t2.id
and t1.age > 20
and t2.hobby like 'S%';
```

Implicit JOIN with Aggregate Functions

```
select t1.department, count(*) as total
from students t1, hobbies t2
where t1.id = t2.id
group by t1.department;
```

Implicit CROSS JOIN (No WHERE)

```
-- Cartesian product (all combinations)
select t1.name, t2.hobby
```

```
from students t1, hobbies t2;
```

Implicit JOIN - Complex Example

```
select t1.name, t2.hobby, t3.course, t4.grade, t5.department_name
from students t1, hobbies t2, courses t3, grades t4, departments t5
where t1.id = t2.student_id
and t1.id = t3.student_id
and t3.id = t4.course_id
and t1.department_id = t5.id
and t4.grade > 80;
```

---

8. NATURAL JOIN

NATURAL JOIN

```
-- Automatically joins on columns with the same name
select *
from students
natural join hobbies;
-- Joins on common column names automatically
```

NATURAL LEFT JOIN

```
select *
from students
natural left join hobbies;
```

NATURAL RIGHT JOIN

```
select *
from students
natural right join hobbies;
```

---

9. JOIN with USING Clause

JOIN with USING

```
-- When both tables have same column name
```

```
select t1.name, t2.hobby
from students t1
join hobbies t2 using (id);
-- Equivalent to: on t1.id = t2.id
```

Multiple Columns with USING

```
select t1.name, t2.hobby
from students t1
join hobbies t2 using (id, department_id);
```

---

10. JOIN Combinations

Mixing JOIN Types

```
-- LEFT JOIN and INNER JOIN together
select t1.name, t2.hobby, t3.course
from students t1
left join hobbies t2 on t1.id = t2.id
inner join courses t3 on t1.id = t3.student_id;
```

Multiple LEFT JOINs

```
select t1.name, t2.hobby, t3.course, t4.grade
from students t1
left join hobbies t2 on t1.id = t2.id
left join courses t3 on t1.id = t3.student_id
left join grades t4 on t3.id = t4.course_id;
```

JOIN with Subquery

```
select t1.name, t2.hobby
from students t1
inner join (
   select * from hobbies where hobby like 'S%'
) t2 on t1.id = t2.id;
```

---

11. JOIN with Different Operators

JOIN with Inequality

```
-- Non-equi join
select t1.name, t2.name
from students t1
join students t2 on t1.age > t2.age;
```

JOIN with BETWEEN

```
select t1.name, t2.salary_range
from employees t1
join salary_grades t2 on t1.salary between t2.min_salary and t2.max_salary;
```

JOIN with OR

```
select t1.name, t2.hobby
from students t1
join hobbies t2 on t1.id = t2.id or t1.department = t2.department;
```

---

12. Advanced JOIN Examples

JOIN with CASE Statement

```
select
   t1.name,
   t2.hobby,
   case
      when t2.hobby is null then 'No Hobby'
      else t2.hobby
   end as hobby_status
from students t1
left join hobbies t2 on t1.id = t2.id;
```

JOIN with GROUP BY and HAVING

```
select t1.department, count(t2.hobby) as hobby_count
from students t1
left join hobbies t2 on t1.id = t2.id
group by t1.department
having count(t2.hobby) > 5;
```

JOIN with ORDER BY

```
select t1.name, t2.hobby
from students t1
inner join hobbies t2 on t1.id = t2.id
order by t1.name, t2.hobby;
```

JOIN with LIMIT

```
select t1.name, t2.hobby
from students t1
inner join hobbies t2 on t1.id = t2.id
limit 10;
```

JOIN with DISTINCT

```
select distinct t1.department
from students t1
inner join hobbies t2 on t1.id = t2.id
where t2.hobby = 'Reading';
```

Comparison: Explicit vs Implicit Joins

Explicit JOIN (Modern, Recommended)

```
-- Clearer, more readable, separates join logic from filtering
select t1.name, t2.hobby
from students t1
inner join hobbies t2 on t1.id = t2.id
where t1.age > 20;
```

Implicit JOIN (Old Style)

```
-- Mixing join and filter conditions in WHERE
select t1.name, t2.hobby
from students t1, hobbies t2
where t1.id = t2.id
and t1.age > 20;
```

Recommendation: Use explicit JOIN syntax (INNER JOIN, LEFT JOIN, etc.) as it's more readable and maintainable!

Quick Reference Summary

JOIN Type	Returns	Syntax
INNER JOIN	Only matching rows from both tables	`t1 INNER JOIN t2 ON condition`
LEFT JOIN	All from left + matching from right	`t1 LEFT JOIN t2 ON condition`
RIGHT JOIN	All from right + matching from left	`t1 RIGHT JOIN t2 ON condition`
FULL OUTER	All rows from both tables	`t1 FULL OUTER JOIN t2 ON condition`
CROSS JOIN	Cartesian product (all combinations)	`t1 CROSS JOIN t2`
SELF JOIN	Table joined to itself	`t1 JOIN t1 AS t2 ON condition`
IMPLICIT	Old style using WHERE	`FROM t1, t2 WHERE condition`
NATURAL	Auto-join on same column names	`t1 NATURAL JOIN t2`

Common Join Patterns

Pattern 1: One-to-Many

```
-- One student, many hobbies
select t1.name, count(t2.hobby) as hobby_count
from students t1
left join hobbies t2 on t1.id = t2.id
group by t1.name;
```

Pattern 2: Many-to-Many (via Junction Table)

```
-- Students and Courses via Enrollments
select t1.name, t3.course_name
from students t1
inner join enrollments t2 on t1.id = t2.student_id
```

```
inner join courses t3 on t2.course_id = t3.id;
```

Pattern 3: Hierarchical (Parent-Child)

```
-- Employees and their managers
select e1.name as employee, e2.name as manager
from employees e1
left join employees e2 on e1.manager_id = e2.id;
```